

Instituto Tecnológico de Buenos Aires

Bases de Datos II

Trabajo Práctico Obligatorio

VetSalud

Grupo 7

Integrantes:

- Rodriguez Acevedo, Clara - 66527
- Buela, Mateo - 64680
- Medina, Sol - 63577

Fecha de entrega: 12 junio 2026

Índice

1. Introducción	1
2. Decisiones de Bases de Datos	2
2.1. Elección de MongoDB	2
2.2. Elección de Redis	2
2.3. División de responsabilidades entre motores	3
3. Modelo de Datos	5
3.1. Modelo en MongoDB	5
3.2. Modelo en Redis	8
3.3. Relación entre ambos motores	9
4. Implementación de las Queries	11
4.1. Queries resueltas con MongoDB	11
4.1.1. Query 1: Pacientes activos con datos de propietario	11
4.1.2. Query 2: Consultas en seguimiento con veterinario y costo	12
4.1.3. Query 3: Historial completo de un paciente	13
4.1.4. Query 4: Propietarios con más de un paciente	15
4.1.5. Query 6: Pacientes con vacunas vencidas	17
4.2. Queries resueltas con Redis	18
4.3. Queries que combinan MongoDB y Redis	18
4.4. Queries cacheadas	18
4.4.1. Query 5: Veterinarios activos con consultas en los últimos 60 días	18
5. Manual de Usuario	20
6. Resultados	21
7. Dificultades y Limitaciones	22
8. Conclusiones	23
9. Referencias	24

1 Introducción

VetSalud es una red ficticia de clínicas veterinarias en Argentina. Este trabajo busca montar un sistema sobre una arquitectura de persistencia polígota con dos motores NoSQL. El sistema gestiona propietarios, sus mascotas o pacientes, veterinarios, consultas y cirugías, vacunaciones, y el stock farmacéutico. Las bases corren en contenedores Docker, y construimos una API REST en Node con Express junto con un dashboard web para ejecutar y visualizar las consultas.

La elección de los dos motores respondió al perfil de los datos. Todo lo clínico y administrativo, es decir propietarios, pacientes, veterinarios, consultas, vacunaciones y catálogo de productos, vive en **MongoDB**. Son datos con esquemas que podrían cambiar con el tiempo y queríamos evitar que migrar fuera dificultoso. Hoy un paciente tiene nombre, especie y raza; mañana podríamos sumar peso, alergias o antecedentes médicos. En MongoDB realizamos consultas analíticas que cruzan entidades, filtran por fecha y agrupan por categoría, usando pipelines del Aggregation Framework con etapas como `$match`, `$lookup`, `$group` y `$project`.

Por otra parte, utilizamos **Redis** para administrar las unidades disponibles del stock farmacéutico. Las unidades de cada producto cambian con cada consulta que consume stock, por lo que necesitábamos operaciones rápidas y atómicas sobre ese contador: leer cuántas unidades hay de un producto, listar productos con poco stock y descontar unidades reduciendo el riesgo de que dos consultas concurrentes dejen el contador en negativo. Redis también cumple un rol de caché de resultados de queries pesadas. Algunas consultas del dashboard agregan sobre toda la colección de consultas y pueden leerse seguido; cachearlas en Redis con TTL corto evita ejecutar la misma agregación todo el tiempo.

El objetivo del trabajo no fue solamente implementar las consultas pedidas, sino también justificar cómo se distribuyen las responsabilidades entre ambos motores y qué ventajas aporta cada uno dentro del sistema. MongoDB concentra el dominio persistente y estructurado de la clínica, mientras que Redis se reserva para información de acceso rápido, contadores y resultados derivados.

2 Decisiones de Bases de Datos

2.1 Elección de MongoDB

Elegimos MongoDB como base principal del dominio clínico y administrativo del sistema. La decisión se apoyó en las características del dominio y en el tipo de consultas que debíamos resolver.

- **Flexibilidad de esquema:** el dominio veterinario puede cambiar con el tiempo. Hoy un paciente tiene nombre, especie, raza y fecha de nacimiento; en una versión futura podría incorporar peso, alergias, antecedentes o alertas médicas. MongoDB permite agregar estos campos sin rediseñar todo el esquema.
- **Datos con estructura de documento:** las entidades principales del sistema tienen varios atributos propios y se representan naturalmente como documentos. Propietarios, pacientes, veterinarios, consultas, vacunaciones y productos no son simples pares clave-valor, sino registros con estructura rica.
- **Consultas analíticas sobre el dominio:** muchas queries requieren filtrar, cruzar entidades, agrupar y proyectar resultados. Para esto usamos pipelines del Aggregation Framework con etapas como `$match`, `$lookup`, `$group`, `$project` y `$sort`.
- **Atomicidad por documento:** las operaciones principales del sistema afectan un documento central por vez, como el alta de un propietario, la modificación de sus datos, la baja lógica de un propietario o la inserción de una consulta. MongoDB garantiza atomicidad sobre cada documento, lo cual alcanza para la mayoría de las operaciones del dominio clínico.
- **Consistencia para datos clínicos y administrativos:** en un backoffice clínico es importante no perder ni duplicar registros. Por eso usamos MongoDB como fuente principal del dominio persistente y centralizamos allí las escrituras de propietarios, pacientes, veterinarios, consultas, vacunaciones y productos.

2.2 Elección de Redis

Elegimos Redis para resolver las partes del sistema que necesitaban acceso rápido, operaciones atómicas simples y expiración automática. En nuestro caso, Redis cumple dos roles: manejar las unidades disponibles del stock farmacéutico y cachear resultados de consultas pesadas.

- **Stock como contador frecuente:** las unidades disponibles de cada producto cambian cada vez que una consulta consume insumos. Este dato se comporta más como

un contador operativo que como un documento clínico, por lo que Redis resulta más adecuado que MongoDB para administrarlo.

- **Lectura rápida de stock puntual:** la lógica de stock necesita consultar cuántas unidades quedan de un producto específico. Redis permite resolver esta lectura de forma directa y con muy baja latencia.
- **Consulta eficiente de stock bajo:** también necesitamos listar productos cuyo stock esté por debajo de cierto umbral. Para eso usamos un *Sorted Set*, donde el identificador del producto es el valor y la cantidad disponible es el score.
- **Decremento atómico:** cuando una consulta consume productos, el decremento individual del contador debe ejecutarse de forma indivisible para evitar resultados negativos. Redis permite validar el stock disponible y descontar unidades de forma atómica mediante un script Lua.
- **Caché con TTL:** algunas queries del dashboard agregan sobre toda la colección de consultas y pueden ejecutarse muchas veces. Redis permite guardar estos resultados con un TTL corto, evitando recalcular agregaciones costosas en cada lectura.

2.3 División de responsabilidades entre motores

La arquitectura quedó dividida según el tipo de dato y el tipo de operación. MongoDB concentra la información persistente y descriptiva del sistema, mientras que Redis almacena datos operativos o derivados que requieren acceso rápido.

- **MongoDB como fuente principal del dominio:** propietarios, pacientes, veterinarios, consultas, vacunaciones y productos viven en MongoDB. Estos datos representan el estado clínico y administrativo de VetSalud.
- **Redis como fuente de verdad del stock:** las unidades disponibles de cada producto viven en Redis. El catálogo del producto está en MongoDB, pero la cantidad disponible se administra en Redis porque cambia con más frecuencia y requiere operaciones atómicas.
- **Redis como caché descartable:** las claves `cache:*` no contienen información primaria, sino resultados derivados de consultas sobre MongoDB. Si una clave expira o se invalida, el resultado puede recalcularse.
- **Separación por prefijos:** las claves de stock usan el prefijo `stock:`, mientras que las claves de caché usan `cache:`. Esto evita mezclar datos con semánticas distintas dentro del mismo motor.

- **Limitación cross-motor:** MongoDB y Redis no comparten una transacción global. Por eso, en operaciones que involucran ambos motores, como registrar una consulta y descontar stock, existe una posible ventana de inconsistencia. Para reducirla, primero se valida stock en Redis, luego se inserta la consulta en MongoDB y finalmente se descuenta el stock de forma atómica.

3 Modelo de Datos

El modelo de datos se diseñó pensando en las consultas que debía resolver el sistema. La idea no fue copiar un modelo relacional tradicional dentro de una base NoSQL, sino repartir la información según el tipo de dato y el tipo de operación esperada. MongoDB concentra las entidades clínicas y administrativas, mientras que Redis guarda estructuras específicas para stock y caché.

El sistema utiliza la base MongoDB `vetsalud`, salvo que se configure otro nombre mediante variables de entorno. Durante la carga inicial, el script de seed lee los archivos CSV, transforma los tipos de datos necesarios y carga las colecciones desde cero. Los identificadores provistos por los datasets se conservan como `_id` de cada documento.

3.1 Modelo en MongoDB

En MongoDB se guardan las entidades principales del dominio de VetSalud: propietarios, pacientes, veterinarios, consultas, vacunaciones y productos. Cada una se representa como una colección independiente porque tiene identidad propia y puede ser consultada desde distintos puntos del sistema.

Las colecciones principales son:

- `propietarios`
- `pacientes`
- `veterinarios`
- `consultas`
- `vacunaciones`
- `productos`

La decisión general fue mantener las relaciones mediante referencias por id. Por ejemplo, un paciente referencia a su propietario con `id_propietario`, y una consulta referencia tanto al paciente como al veterinario mediante `id_paciente` e `id_vet`. Esto permite mantener las entidades separadas y evitar duplicar información descriptiva, pero al mismo tiempo deja disponible la posibilidad de cruzarlas mediante `$lookup` cuando una query necesita información combinada.

Los propietarios contienen la información de contacto y ubicación de cada cliente de la clínica. Además, incluyen un campo `activo`, utilizado para implementar baja lógica. Esta decisión permite conservar los datos históricos aunque un propietario deje de estar activo.

Ejemplo de propietario

```
1 {
2   "_id": "C001",
3   "nombre": "Ana",
4   "apellido": "Rodriguez",
5   "dni": "30456789",
6   "email": "ana@gmail.com",
7   "telefono": "1145001234",
8   "ciudad": "Buenos Aires",
9   "provincia": "Buenos Aires",
10  "activo": true
11 }
```

Los pacientes representan las mascotas atendidas por VetSalud. Cada paciente referencia a su propietario, pero no se encuentra embebido dentro del documento del dueño. Elegimos esta separación porque un propietario puede tener varios pacientes y porque muchas consultas necesitan trabajar directamente sobre la colección de pacientes. El campo `fecha_nac` se carga como fecha en MongoDB.

Ejemplo de paciente

```
1 {
2   "_id": "P001",
3   "nombre": "Firulais",
4   "especie": "Perro",
5   "raza": "Labrador",
6   "fecha_nac": "2019-04-12T00:00:00.000Z",
7   "id_propietario": "C001",
8   "activo": true
9 }
```

Los veterinarios se modelan como una colección propia porque son una entidad central para varias consultas del sistema. Además de los datos profesionales, cada veterinario tiene una sucursal asociada. Esta decisión es importante porque algunas queries necesitan vincular pacientes o atenciones con la sucursal del veterinario que las realizó.

Ejemplo de veterinario

```
1 {
2   "_id": "V001",
3   "nombre": "Gabriela",
```



```

4  "apellido": "Fontana",
5  "matricula": "VET-0041",
6  "especialidad": "Clinica General",
7  "sucursal": "Palermo",
8  "activo": true
9  }

```

Una de las decisiones más importantes del modelo fue guardar consultas y cirugías en una misma colección llamada `consultas`. Ambas representan atenciones clínicas y comparten la mayor parte de sus campos: paciente, veterinario, fecha, motivo, diagnóstico, costo y estado. Para diferenciarlas se utiliza el campo `tipo`. Esto evita duplicar pipelines y simplifica las queries que trabajan sobre historial clínico, diagnósticos frecuentes o ingresos por veterinario.

Ejemplo de consulta

```

1  {
2    "_id": "CON001",
3    "id_paciente": "P001",
4    "id_vet": "V001",
5    "fecha": "2025-10-05T00:00:00.000Z",
6    "tipo": "Consulta",
7    "motivo": "Control anual",
8    "diagnostico": "Sano",
9    "costo": 4500,
10   "estado": "Cerrada"
11 }

```

Las vacunaciones se guardan en una colección separada. Aunque también se relacionan con pacientes y veterinarios, su estructura tiene otra semántica: lo central es la vacuna aplicada y la fecha de próxima dosis. Mezclarlas con consultas hubiera generado documentos con campos opcionales y no aportaba una ventaja clara para las queries pedidas. Los campos `fecha_aplicacion` y `proxima_dosis` se cargan como fechas.

Ejemplo de vacunación

```

1  {
2    "_id": "VAC001",
3    "id_paciente": "P001",
4    "id_vet": "V001",
5    "fecha_aplicacion": "2025-10-05T00:00:00.000Z",

```

```

6   "nombre_vacuna": "Antirrabica",
7   "proxima_dosis": "2026-10-05T00:00:00.000Z"
8 }

```

Por último, la colección `productos` guarda el catálogo farmacéutico. En MongoDB se conserva la información descriptiva y relativamente estable de cada producto, como nombre, categoría, precio unitario, vencimiento y proveedor. La cantidad disponible no se guarda en esta colección porque forma parte del stock operativo, que cambia con mayor frecuencia y se administra en Redis.

Ejemplo de producto

```

1 {
2   "_id": "PRD001",
3   "nombre": "Amoxicilina 250mg",
4   "categoria": "Antibiotico",
5   "precio_unit": 850,
6   "vencimiento": "2026-06-30T00:00:00.000Z",
7   "proveedor": "VetFarma SA"
8 }

```

Además de las colecciones, se definieron índices sobre campos usados frecuentemente en filtros, relaciones o rangos. El seed crea índices sobre `fecha` e `id_vet` en `consultas`, sobre `id_propietario` en `pacientes`, y sobre `proveedor` y `vencimiento` en `productos`. Estos índices acompañan las consultas implementadas y evitan depender únicamente de recorridos completos sobre las colecciones.

3.2 Modelo en Redis

Redis se usa con un modelo más acotado que MongoDB. No se intentó replicar las entidades clínicas ni duplicar el dominio completo, sino guardar únicamente estructuras donde Redis aporta una ventaja clara: unidades disponibles del stock farmacéutico y caché de consultas pesadas.

La estructura principal es `stock:unidades`, un *Sorted Set* donde cada producto tiene como valor su identificador y como score la cantidad disponible. Esta estructura se carga desde el campo `unidades` del CSV de stock farmacéutico.

Estructura de stock en Redis

```

1 stock:unidades

```

```
2
3 value    score
4 PRD001   120
5 PRD004   40
6 PRD006   25
7 PRD017   15
```

El uso de un *Sorted Set* responde directamente a la necesidad de ordenar y filtrar por cantidad disponible. Para la consulta de stock bajo, Redis permite buscar por rango sobre el score y obtener rápidamente los productos por debajo de un umbral.

Consulta de stock bajo

```
1 ZRANGEBYSCORE stock:unidades -inf 49
```

El descuento de stock es el caso más sensible porque involucra concurrencia sobre un contador compartido. Por eso, la lógica de decremento se ejecuta dentro de Redis mediante un script Lua atómico, que verifica existencia, valida cantidad suficiente y descuenta en una única operación indivisible.

Además del stock, Redis guarda resultados cacheados de algunas queries del dashboard. Estas claves siguen el prefijo `cache:` y contienen resultados serializados en JSON con un TTL corto. Esto permite acelerar consultas frecuentes sin convertir esos resultados en fuente de verdad.

Las claves de caché usadas por el sistema son:

- `cache:vets-consultas-60d`, con TTL de 300 segundos.
- `cache:top-diagnosticos`, con TTL de 300 segundos.
- `cache:ingresos-vet-mes`, con TTL de 60 segundos.

Estas claves representan información derivada. Si expiran o se invalidan, el resultado puede recalcularse desde MongoDB. Por eso tienen una semántica distinta a `stock:unidades`: el stock representa estado operativo del sistema, mientras que la caché representa resultados temporales.

3.3 Relación entre ambos motores

La relación entre MongoDB y Redis aparece principalmente en el manejo del stock farmacéutico. MongoDB guarda el catálogo de productos, mientras que Redis guarda las unidades disponibles. Esto significa que para mostrar productos con stock bajo el sistema

combina información de ambos motores: Redis identifica rápidamente qué productos están por debajo del umbral y MongoDB aporta los datos descriptivos de esos productos.

Este diseño evita duplicar información descriptiva. Redis no necesita saber el nombre, proveedor o vencimiento de cada producto; solo necesita saber cuántas unidades quedan. MongoDB, por su parte, no necesita actualizar constantemente un contador de stock que cambia con cada consulta. Cada motor conserva la parte del dato que mejor se adapta a su modelo.

La relación también aparece en las queries cacheadas. En esos casos, MongoDB sigue siendo la fuente de verdad: la query se calcula sobre sus colecciones y el resultado se guarda temporalmente en Redis. Redis no reemplaza a MongoDB, sino que funciona como una capa de aceleración para lecturas frecuentes.

Esta separación tiene una limitación: no existe una transacción global entre ambos motores. Por ejemplo, al registrar una consulta que consume productos, el sistema debe insertar el documento de la consulta en MongoDB y descontar unidades en Redis. Para reducir el riesgo de inconsistencias, se valida el stock antes de insertar la consulta y se descuenta mediante una operación atómica en Redis. Sin embargo, la coordinación entre ambos motores sigue siendo una limitación inherente de la arquitectura elegida.

4 Implementación de las Queries

4.1 Queries resueltas con MongoDB

4.1.1 Query 1: Pacientes activos con datos de propietario

La primera query expone el endpoint `GET /api/pacientes-activos` y resuelve el listado de pacientes activos junto con los datos completos de su propietario. Se implementa sobre MongoDB utilizando las colecciones `pacientes` y `propietarios`.

La consulta primero filtra los documentos de `pacientes` cuyo campo `activo` vale `true`. Luego utiliza `$lookup` para unir cada paciente con su propietario, comparando `pacientes.id_propietario` contra `propietarios._id`. Como cada paciente tiene un único propietario, se aplica `$unwind` sobre el resultado de la unión y finalmente se proyectan los campos relevantes.

Esta query aprovecha directamente la decisión de modelar pacientes y propietarios como colecciones separadas vinculadas por referencia. De esta forma, no se duplica la información del propietario dentro de cada paciente, pero se puede reconstruir el resultado combinado cuando el caso de uso lo requiere.

La proyección incluye los datos del paciente activo y todos los campos del propietario cargados por el seed: identificador, nombre, apellido, DNI, email, teléfono, ciudad, provincia y estado lógico.

Ejemplo de resultado de Query 1

```
1  [
2    {
3      "_id": "P005",
4      "nombre": "Rex",
5      "especie": "Perro",
6      "raza": "Pastor Aleman",
7      "fecha_nac": "2020-09-08T00:00:00.000Z",
8      "activo": true,
9      "propietario": {
10       "_id": "C004",
11       "nombre": "Diego",
12       "apellido": "Herrera",
13       "dni": "35789012",
14       "email": "diego@gmail.com",
15       "telefono": "1178004567",
16       "ciudad": "Mendoza",
```

```

17     "provincia": "Mendoza",
18     "activo": true
19   }
20 }
21 ]

```

4.1.2 Query 2: Consultas en seguimiento con veterinario y costo

La segunda query expone el endpoint `GET /api/consultas-seguimiento` y lista las consultas médicas abiertas cuyo estado es `Seguimiento`. Se resuelve íntegramente en MongoDB usando las colecciones `consultas`, `veterinarios` y `pacientes`.

El pipeline comienza con un `$match` sobre `consultas.estado` para quedarse únicamente con las atenciones en seguimiento. Luego realiza un `$lookup` contra `veterinarios` usando `id_vet`, lo que permite mostrar el veterinario asignado a la atención. Después hace un segundo `$lookup` contra `pacientes` para incorporar datos básicos del animal atendido. En ambos casos se usa `$unwind`, porque cada consulta referencia a un único veterinario y a un único paciente.

La proyección final conserva los datos principales de la consulta, incluyendo fecha, tipo, motivo, diagnóstico, estado y costo. También incluye el identificador del paciente, un bloque con datos básicos del paciente y otro bloque con los datos profesionales del veterinario. El resultado se ordena por fecha descendente para mostrar primero las atenciones más recientes que todavía requieren seguimiento.

Esta query aprovecha el campo `estado` de la colección `consultas`, que permite distinguir atenciones cerradas de atenciones abiertas. También aprovecha que las atenciones clínicas, incluidas las cirugías, comparten una misma colección: el seguimiento operativo se puede consultar desde `consultas` sin duplicar lógica en otra entidad.

Ejemplo de resultado de Query 2

```

1  [
2    {
3      "_id": "CON004",
4      "id_paciente": "P004",
5      "fecha": "2025-11-01T00:00:00.000Z",
6      "tipo": "Consulta",
7      "motivo": "Perdida de apetito",
8      "diagnostico": "Estres ambiental",
9      "costo": 3800,
10     "estado": "Seguimiento",

```

```

11     "paciente": {
12         "_id": "P004",
13         "nombre": "Nube",
14         "especie": "Conejo",
15         "raza": "Angora"
16     },
17     "veterinario": {
18         "_id": "V001",
19         "nombre": "Gabriela",
20         "apellido": "Fontana",
21         "matricula": "VET-0041",
22         "especialidad": "Clinica General",
23         "sucursal": "Palermo"
24     }
25 }
26 ]

```

4.1.3 Query 3: Historial completo de un paciente

La tercera query expone el endpoint `GET /api/historial-paciente/:id` y reconstruye el historial clínico de un paciente a partir de sus consultas y vacunaciones. Se resuelve principalmente con MongoDB, usando las colecciones `pacientes`, `consultas`, `vacunaciones` y `veterinarios`, y luego combina los resultados en Node.js.

Antes de ejecutar los pipelines, la función valida que el paciente solicitado exista. Si el identificador no corresponde a ningún documento de `pacientes`, se devuelve un error. Esta validación evita construir historiales sobre pacientes inexistentes.

La implementación ejecuta dos agregaciones en paralelo. La primera consulta la colección `consultas`, filtra por `id_paciente`, une cada atención con su veterinario mediante `$lookup` y proyecta el evento con un formato común. La segunda hace lo mismo sobre `vacunaciones`, usando `fecha_aplicacion` como fecha del evento e incorporando los campos propios de vacunas, como `nombre_vacuna` y `proxima_dosis`.

Se decidió realizar la combinación final en la capa de servicio porque el historial integra eventos de dos colecciones con estructuras distintas. MongoDB concentra el trabajo más pesado de cada origen: filtra por paciente, resuelve la unión con veterinarios y proyecta cada evento con los campos necesarios. Node.js queda limitado a una tarea de orquestación: llevar ambas listas a un formato común, unirlas y aplicar el criterio de ordenamiento final.

Ambos resultados se normalizan a una misma estructura con campos como `tipo_evento`, `id_evento`, `fecha`, `paciente` y `veterinario`. Luego se concatenan en Node.js y se or-

denan por fecha descendente, mostrando primero los eventos más recientes. En caso de empate de fecha, se usa el identificador del evento como segundo criterio de orden.

Esta query justifica dos decisiones de modelado. Por un lado, consultas y cirugías comparten la colección `consultas`, por lo que todas las atenciones clínicas del paciente se recuperan desde el mismo origen. Por otro lado, vacunaciones se mantiene como colección separada porque tiene una semántica distinta, pero puede integrarse al historial mediante una normalización final del resultado.

Fragmento de resultado de Query 3 para P001

```
1  [
2    {
3      "tipo_evento": "Consulta",
4      "id_evento": "CON007",
5      "fecha": "2025-12-02T00:00:00.000Z",
6      "paciente": {
7        "_id": "P001",
8        "nombre": "Firulais",
9        "especie": "Perro",
10       "raza": "Labrador"
11     },
12     "veterinario": {
13       "_id": "V001",
14       "nombre": "Gabriela",
15       "apellido": "Fontana",
16       "matricula": "VET-0041",
17       "sucursal": "Palermo"
18     },
19     "motivo": "Control post-vacuna",
20     "diagnostico": "Sin novedades",
21     "costo": 2000,
22     "estado": "Cerrada",
23     "nombre_vacuna": null,
24     "proxima_dosis": null
25   },
26   {
27     "tipo_evento": "Vacunacion",
28     "id_evento": "VAC001",
29     "fecha": "2025-10-05T00:00:00.000Z",
30     "paciente": {
31       "_id": "P001",
```



```

32     "nombre": "Firulais",
33     "especie": "Perro",
34     "raza": "Labrador"
35 },
36 "veterinario": {
37     "_id": "V001",
38     "nombre": "Gabriela",
39     "apellido": "Fontana",
40     "matricula": "VET-0041",
41     "sucursal": "Palermo"
42 },
43 "motivo": null,
44 "diagnostico": null,
45 "costo": null,
46 "estado": null,
47 "nombre_vacuna": "Antirrabica",
48 "proxima_dosis": "2026-10-05T00:00:00.000Z"
49 }
50 ]

```

4.1.4 Query 4: Propietarios con más de un paciente

La cuarta query expone el endpoint `GET /api/proprietarios-multiples-pacientes` y lista los propietarios que tienen más de un paciente registrado. Se implementa sobre MongoDB usando las colecciones `pacientes` y `proprietarios`.

El pipeline comienza desde `pacientes`, porque el dato que se necesita contar es la cantidad de mascotas asociadas a cada propietario. Primero agrupa por `id_propietario`, calcula `cantidad_pacientes` con `$sum` y guarda en un arreglo los datos básicos de cada paciente mediante `$push`. Luego aplica un `$match` para conservar únicamente los grupos con más de un paciente.

Después de identificar los propietarios que cumplen la condición, la query usa `$lookup` contra `proprietarios` para incorporar sus datos de contacto. Como cada grupo corresponde a un único propietario, se aplica `$unwind` y finalmente se proyecta una salida con el identificador del propietario, la cantidad total de pacientes, los datos del propietario y el detalle de las mascotas asociadas.

La consulta cuenta pacientes registrados, no solamente pacientes activos, porque la consigna no restringe este punto a mascotas activas. Por eso el resultado conserva el campo `activo` en cada paciente: permite ver si alguna de las mascotas asociadas está dada de baja lógicamente sin excluirla del conteo. El resultado se ordena por cantidad de

pacientes en forma descendente y, ante empate, por identificador de propietario.

Ejemplo de resultado de Query 4

```
1  [
2    {
3      "cantidad_pacientes": 3,
4      "pacientes": [
5        {
6          "_id": "P001",
7          "nombre": "Firulais",
8          "especie": "Perro",
9          "raza": "Labrador",
10         "activo": true
11       },
12       {
13         "_id": "P003",
14         "nombre": "Coco",
15         "especie": "Perro",
16         "raza": "Beagle",
17         "activo": false
18       },
19       {
20         "_id": "P018",
21         "nombre": "Pelusa",
22         "especie": "Conejo",
23         "raza": "Holland Lop",
24         "activo": true
25       }
26     ],
27     "propietario": {
28       "_id": "C001",
29       "nombre": "Ana",
30       "apellido": "Rodriguez",
31       "email": "ana@gmail.com",
32       "telefono": "1145001234",
33       "ciudad": "Buenos Aires",
34       "provincia": "Buenos Aires",
35       "activo": true
36     },
37     "id_propietario": "C001"
38   }
```

4.1.5 Query 6: Pacientes con vacunas vencidas

La sexta query expone el endpoint `GET /api/pacientes-vacunas-vencidas` y lista los pacientes que tienen al menos una vacunación cuya próxima dosis es anterior a la fecha actual. Se resuelve íntegramente en MongoDB usando las colecciones `vacunaciones` y `pacientes`.

El pipeline parte de `vacunaciones`, porque el criterio de vencimiento está en el campo `proxima_dosis`. La función toma la fecha del sistema, la normaliza al inicio del día y filtra con `$match` las vacunaciones donde `proxima_dosis < hoy`. Esta normalización evita marcar como vencida una vacuna cuya próxima dosis sea el mismo día de ejecución.

Después, la query agrupa por paciente. Para cada uno calcula la cantidad de vacunas vencidas, conserva el detalle en un arreglo y obtiene la próxima dosis más antigua con `$min`. Luego realiza un `$lookup` contra `pacientes` para incorporar los datos básicos del animal y proyecta el resultado final.

El ordenamiento usa primero `proxima_dosis_mas_antigua` en forma ascendente, de modo que los pacientes con vencimientos más antiguos aparezcan antes. Esto permite usar la consulta como una lista de priorización operativa para contactar propietarios o programar revacunaciones.

Esta query justifica mantener las vacunaciones como colección separada de las consultas. El vencimiento de una vacuna no depende del diagnóstico ni del costo de una atención, sino de una fecha propia del calendario sanitario del paciente.

Ejemplo de resultado de Query 6

```

1  [
2    {
3      "cantidad_vacunas_vencidas": 2,
4      "vacunas_vencidas": [
5        {
6          "id_vacuna": "VAC022",
7          "nombre_vacuna": "Sextuple",
8          "fecha_aplicacion": "2024-06-10T00:00:00.000Z",
9          "proxima_dosis": "2025-06-10T00:00:00.000Z",
10         "id_vet": "V001"
11       },
12       {
13         "id_vacuna": "VAC031",

```

```

14     "nombre_vacuna": "Triple Felina",
15     "fecha_aplicacion": "2024-04-15T00:00:00.000Z",
16     "proxima_dosis": "2025-04-15T00:00:00.000Z",
17     "id_vet": "V001"
18   }
19 ],
20 "proxima_dosis_mas_antigua": "2025-04-15T00:00:00.000Z",
21 "paciente": {
22   "_id": "P003",
23   "nombre": "Coco",
24   "especie": "Perro",
25   "raza": "Beagle",
26   "activo": false
27 }
28 }
29 ]

```

4.2 Queries resueltas con Redis

4.3 Queries que combinan MongoDB y Redis

4.4 Queries cacheadas

4.4.1 Query 5: Veterinarios activos con consultas en los últimos 60 días

La quinta query expone el endpoint `GET /api/vets-activos-consultas-60d` y lista los veterinarios activos junto con la cantidad de consultas registradas en los últimos 60 días. Se calcula con MongoDB sobre las colecciones `veterinarios` y `consultas`, y el resultado se guarda temporalmente en Redis como caché.

La función toma la fecha del sistema y construye un rango de días calendario: desde el inicio del día ubicado 60 días atrás hasta el inicio del día siguiente al de ejecución. Este criterio es importante porque los datos del proyecto se cargan como fechas de día completo, no como instantes con hora clínica.

El pipeline comienza desde `veterinarios` con un `$match` sobre `activo: true`. Luego usa un `$lookup` con pipeline correlacionado contra `consultas`: compara el veterinario de cada atención con el documento activo que se está procesando y filtra la fecha dentro del rango calculado. En la proyección final, la cantidad se obtiene con `$size` sobre el arreglo de consultas recientes.

El resultado incluye también veterinarios activos sin consultas recientes, porque

la consigna pide listar veterinarios activos y su cantidad de consultas. Finalmente se ordena por `cantidad_consultas_60d` en forma descendente y, ante empates, por apellido y nombre.

Esta query es buena candidata para caché porque cruza veterinarios con consultas y puede ejecutarse repetidamente desde el dashboard. Por eso se guarda en Redis bajo la clave `cache:vets-consultas-60d` con un TTL de 300 segundos. Cuando se registra una nueva consulta médica, la función de alta invalida esta clave para que el próximo pedido recalculé el resultado desde MongoDB.

Ejemplo de resultado de Query 5

```
1  [  
2    {  
3      "nombre": "Gabriela",  
4      "apellido": "Fontana",  
5      "matricula": "VET-0041",  
6      "especialidad": "Clinica General",  
7      "sucursal": "Palermo",  
8      "activo": true,  
9      "id_vet": "V001",  
10     "cantidad_consultas_60d": 8  
11   }  
12 ]
```

5 Manual de Usuario

6 Resultados

7 Dificultades y Limitaciones

8 Conclusiones

9 Referencias

- Documentación oficial de MongoDB: <https://www.mongodb.com/docs/>
- Documentación oficial de Redis: <https://redis.io/docs/latest/>
- Documentación de Docker: <https://docs.docker.com/>